ADVANCED OBJECT-ORIENTED PROGRAMMING USING JAVA

CO-1: TEAM-BASED CASE STUDY EXERCISES

CASE STUDY 3
BANK ACCOUNT AND MONEY TRANSFERS

Name	V.GAUTAM
Register Number	2520030316
Class / Section	Section-1 cluster 1

Based on the Case Study 3 questions in the supplied CO-1 assignment.

CASE STUDY 3 – PROBLEM STATEMENT

A retail banking organization wants to develop a Java-based banking application that allows customers to maintain accounts, deposit money, withdraw money and transfer funds between accounts. The existing system has problems such as withdrawals beyond overdraft limits, incomplete transfers, withdrawals from frozen accounts, incorrect balances and transactions with zero or negative amounts.

Q1. Identification of Domain Invariants (6 Marks)

Domain invariants are business rules that must always be true in the banking application.

1. **Positive transaction amount:** Deposit, withdrawal and transfer amounts must be greater than zero. This prevents meaningless transactions and incorrect balance changes.
2. **Overdraft limit:** An account balance must not go below the permitted overdraft limit. Otherwise, customers may withdraw more money than allowed.
3. **Frozen account restriction:** A frozen account must not allow withdrawals. If this rule is violated, a blocked account could still lose money.
4. **Closed account restriction:** A closed account must not accept normal deposits or withdrawals. Otherwise, transactions may be recorded against an inactive account.
5. **Atomic fund transfer:** A transfer must debit the source and credit the destination together. If only one side succeeds, money may appear to be lost.
6. **Balance and ledger consistency:** The account balance must agree with its transaction records. If they differ, customers and the bank may see incorrect financial information.

Conclusion: These invariants protect customer money, prevent invalid operations and keep banking records accurate.

Q2. Entities and Value Objects (6 Marks)

An entity has a unique identity and may change during its lifetime. A value object is identified by its value and is usually immutable.

Concept	Classification	Justification

Bank Account	Entity	Has a unique account identity; its balance and status can change.
Account Number	Value Object	Its value identifies an account and should not be changed.
Money	Value Object	Defined by amount and currency; equal values represent equal money.
Ledger Entry	Entity	Each entry has a unique transaction identity and a recorded lifecycle.
Overdraft Limit	Value Object	Defined by its monetary amount and currency.
Fund Transfer	Entity	Has a unique transfer ID and a status such as pending or completed.

Conclusion: Entities are tracked by identity, while value objects are compared by their values.

Q3. Self-Enforcing Objects and Java Design (8 Marks)

A self-enforcing object checks its own business rules before allowing an operation. The BankAccount class keeps its fields private and provides controlled methods.

Java Program

```
import java.math.BigDecimal;

class BankAccount {
    private String accountNumber;
    private BigDecimal balance;
    private BigDecimal overdraftLimit;
    private String status;

    public BankAccount(String accountNumber, BigDecimal balance,
                       BigDecimal overdraftLimit) {
        if (balance == null || overdraftLimit == null ||
            balance.compareTo(BigDecimal.ZERO) < 0 ||
            overdraftLimit.compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException("Invalid opening values");
        }
        this.accountNumber = accountNumber;
        this.balance = balance;
        this.overdraftLimit = overdraftLimit;
        this.status = "ACTIVE";
    }

    public void deposit(BigDecimal amount) {
        if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0)
            throw new IllegalArgumentException("Amount must be positive");
        if (status.equals("CLOSED"))
            throw new IllegalStateException("Account is closed");
        balance = balance.add(amount);
        System.out.println("Deposit successful");
    }

    public void withdraw(BigDecimal amount) {
        if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0)
            throw new IllegalArgumentException("Amount must be positive");
        if (!status.equals("ACTIVE"))
            throw new IllegalStateException("Account is not active");

        BigDecimal newBalance = balance.subtract(amount);
        if (newBalance.compareTo(overdraftLimit.negate()) < 0)
```

```
            throw new IllegalStateException("Overdraft limit exceeded");

        balance = newBalance;
        System.out.println("Withdrawal successful");
    }

    public void freeze() {
        if (status.equals("CLOSED"))
            throw new IllegalStateException("Account is closed");
        status = "FROZEN";
        System.out.println("Account frozen");
    }

    public void unfreeze() {
        if (!status.equals("FROZEN"))
            throw new IllegalStateException("Account is not frozen");
        status = "ACTIVE";
        System.out.println("Account unfrozen");
    }

    public void close() {
        if (status.equals("CLOSED"))
            throw new IllegalStateException("Account already closed");
        status = "CLOSED";
        System.out.println("Account closed");
    }

    public BigDecimal getBalance() {
        return balance;
    }
}

public class BankDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount(
            "101", new BigDecimal("10000"), new BigDecimal("2000"));

        account.deposit(new BigDecimal("2000"));
        account.withdraw(new BigDecimal("3000"));
        System.out.println("Balance: " + account.getBalance());

        account.freeze();
        account.unfreeze();
        account.close();
    }
}
```

Program Explanation

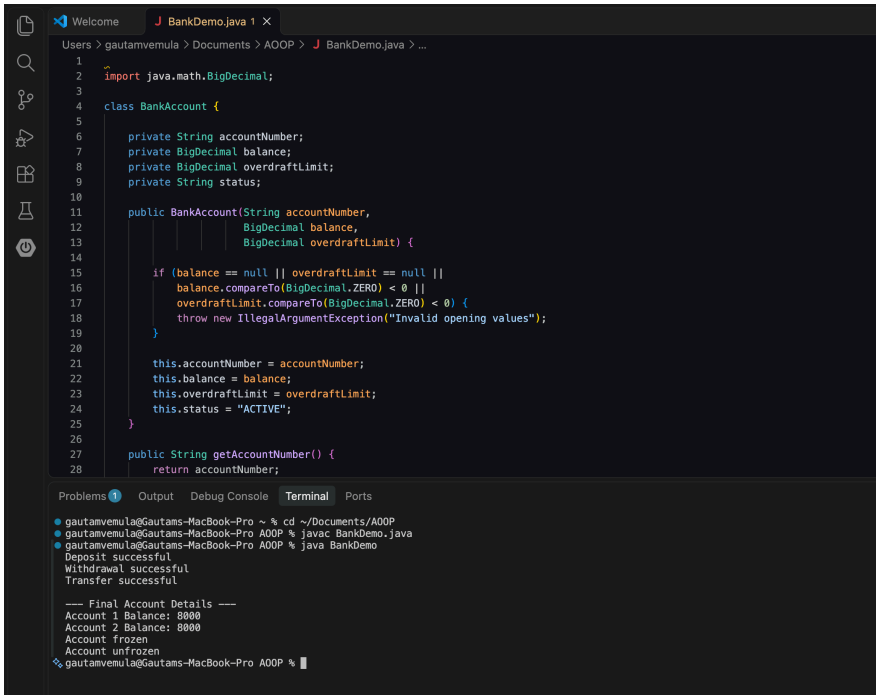
- The BankAccount class stores the account number, balance, overdraft limit and status.
- deposit() accepts only a positive amount and does not allow deposits into a closed account.
- withdraw() checks the account status, amount and overdraft limit before changing the balance.
- freeze() blocks normal account operations; unfreeze() changes a frozen account back to active.
- close() changes the account status to closed.
- Private fields prevent other classes from directly changing the account data.

Expected Output

```
Deposit successful
Withdrawal successful
Balance: 9000
Account frozen
```

```
Account unfrozen
Account closed
```

Paste a screenshot of the successful Java execution here



How the Program Prevents Errors

- Rejects zero and negative transaction amounts.
- Does not allow withdrawals from frozen or closed accounts.
- Checks the overdraft limit before completing a withdrawal.
- Uses private fields so external classes cannot directly change the balance.

Note: This is a teaching example for account operations. A real banking application also needs persistent ledger entries, secure authorization and database transactions.

Q4. Money Transfer and Aggregate Consistency (6 Marks)

Example: A customer transfers ₹5,000 from Account A to Account B. The system deducts the money from A but crashes before crediting B.

1. Problems that can arise

- Money is deducted from Account A but Account B does not receive it.
- The account balances and transaction history may become incorrect.
- The customer may lose access to the transferred money until the problem is corrected.

2. Invariants violated

- A transfer must be completed on both accounts or not completed at all.
- The account balance must agree with the ledger and transaction records.

3. Design to prevent money loss

- Perform the debit and credit as one database transaction.
- Use ACID properties so both changes succeed together or are rolled back.

4. Handling failures

- Begin the database transaction.
- Check that the source account is active and has sufficient permitted funds.
- Debit ₹5,000 from Account A.
- Credit ₹5,000 to Account B.
- Record both sides of the transfer in the ledger.
- Commit if all operations succeed; otherwise rollback all changes.

5. Accounts in different banking systems

- A single local database transaction may not cover both banks.
- Use a reliable distributed transfer workflow with a unique transaction ID, acknowledgements and status tracking.
- Retry safely or reverse/compensate a failed transfer so money is not duplicated or lost.

ACID properties:

- Atomicity – Either all operations happen or none happen.
- Consistency – Data follows the banking rules.
- Isolation – Concurrent transactions do not interfere incorrectly.
- Durability – Committed changes remain saved.

Q5. Ledger Management and Design Evaluation (4 Marks)

1. Information in a ledger entry: Transaction ID, account number, transaction type, amount, date and time, and transaction status.

2. Modify or delete completed entries?: No. Completed ledger entries should not be changed or deleted because they are permanent evidence of transactions. Corrections should be made using a new reversal or adjustment entry.

3. Keep balance consistent with history: Record every transaction, update the balance through controlled operations, use database transactions and regularly reconcile the account balance with ledger entries.

4. Challenges with very large history: Searching and reporting may become slower, storage requirements increase, and backups may take longer. Indexing, pagination and archiving older records can help.

Quick Revision – Important Terms

Term	Simple meaning
Invariant	A business rule that must always be true.
Entity	An object with a unique identity.
Value Object	An object identified by its values.
Self-enforcing object	An object that checks and protects its own rules.
Rollback	Undoing changes when a database transaction fails.
Ledger	A record of account transactions.
ACID	Atomicity, Consistency, Isolation and Durability.

END OF CASE STUDY 3